Python Assignment: Topic-wise (Based on Python Complete Notes)

Name: _____ Roll No: _____ Date: _____

General Instructions

1. Do the sections in order. Each section matches one page of the Python Complete Notes PDF.
2. Save each section in its own file, named `section01.py`, `section02.py`, and so on.
3. Write each question's number as a comment (`# Q1`) above its code.
4. Where a question shows an output, your program must print exactly that output.
5. Run every program before submitting. Paste a sample output at the bottom of each file as a comment.
6. Where input is asked, take it using `input()`.

Section 1: Introduction to Python (Notes Page 1)

Q1. Open a terminal or command prompt and run `python --version`. Write the version you see in a comment.

Q2. Create a file `first.py`. Write one line that prints `Hello, World!`. Run it from the terminal using `python first.py`.

Q3. In `first.py`, add a `#` comment above the print line saying what the line does.

Q4. Write a program that prints your name, your class, and your city, each on a separate line, using three `print()` statements.

Q5. In 4 to 5 lines (as comments in your file), write the steps of the Python Execution Flow: Write Code, Interpreter, Execute, Output.

Q6. Write 3 features of Python from the notes (for example, Interpreted, Cross-Platform, Large Standard Library) and give a one-line meaning of each as comments.

Section 2: Variables & Data Types (Notes Page 2)

Q1. Create four variables: `name = 'Ravi'`, `age = 20`, `price = 19.99`, `is_active = True`. Print each value on its own line.

Q2. For the four variables in Q1, print the type of each using `type()`. Expected first line: `<class 'str'>`.

Q3. Check these variable names and write `Valid` or `Invalid` beside each as a comment: `my_var`, `2value`, `user-name`, `_count`, `my var`, `class`, `total2`. Write a one-line reason for each invalid name.

Q4. Create `a = 10`, `b = -7`, `c = 3.1415`. Print the type of each variable.

Q5. Create the strings `s1` using single quotes, `s2` using double quotes, and `s3` using triple quotes across two lines. Print all three.

Q6. Convert the string `'10'` to an int and the string `'3.14'` to a float. Print both with their types.

Q7. Convert the number `100` into a string, then print `bool(0)` and `bool(1)`. Write a comment saying what each prints.

Q8. Create one example each of list, tuple, set, and dict (3 items each). Print the type of each.

Q9. Create `s = 'hello'` and try `s[0] = 'H'`. Write a comment explaining the error you get and why it happens (strings are immutable).

Section 3: Input, Output & Operators (Notes Page 3)

Q1. Ask the user for their name using `input()`. Print `Hello, <name>!`.

Q2. Ask the user for their age, then print `type(age)`. Write a comment explaining why the type is `str`.

Q3. Print `A`, `B`, `C` using one `print()` so the output is `A-B-C`. (Hint: use `sep`.)

Q4. Use `end` to print `Hello` and `World` on the same line with a space between them, using two separate `print()` statements.

Q5. Set `x = 10` and print `Value of x is 10` using an f-string.

Q6. Take two numbers from the user. Print their sum, difference, product, division, floor division, modulus, and power. Label each, e.g. `Sum = 15`.

Q7. Take two numbers. Print the result (`True` / `False`) of `==`, `!=`, `>`, `<`, `>=`, `<=`.

Q8. Take a number. Print `True` if it is greater than 3 and less than 10 (use `and`). Then print `True` if it is less than 3 or greater than 10 (use `or`). Then print `not` of the first result.

Q9. Set `x = 10`. Apply `+= 5`, `-= 3`, `*= 2`, `/= 4`, `//= 2`, `%= 3`, `**= 2` one after another, printing `x` after each.

Q10. For `lst = [1, 2, 3, 4]`, print whether `2` is in `lst` and whether `5` is not in `lst`.

Q11. Create `a = [1, 2, 3]`, `b = a`, `c = [1, 2, 3]`. Print `a is b`, `a is c`, and `a == c`. Write a comment explaining why the results differ.

Q12. Without running it, write the answer of `10 + 5 * 2 ** 2` as a comment. Then run it and check.

Section 4: Decision Making (Notes Page 4)

Q1. Take an age. If it is 18 or more, print `You are eligible to vote.`

Q2. Take a number. Print `Positive`, `Negative`, or `Zero` using `if-elif-else`.

Q3. Take marks out of 100. Print grade `A` for 90 and above, `B` for 60 to 89, and `C` below 60.

Q4. Take a number and print `Even` or `Odd`.

Q5. Take an age and ask `Do you have an ID? (yes/no)`. Using a nested `if`: if age is 18 or more and the answer is `yes`, print `Entry Allowed`. If age is 18 or more but the answer is `no`, print `ID is required`. Otherwise print `Not eligible`.

Q6. Take three numbers and print the largest using `if-elif-else`.

Q7. Take a day number from 1 to 7. Using `match-case` (Python 3.10+), print `Monday` to `Sunday`. For 6 and 7 use `case 6 | 7` to print `Weekend`. Use `case _` to print `Invalid day`.

Q8. Login system: ask for a username and a password. If the username is `admin` and the password is `1234`, print `Login Successful`. If the username is wrong, print `Invalid Username`. If only the password is wrong, print `Incorrect Password`. Use nested `if`.

Section 5: Loops (Notes Page 5)

Q1. Using a `while` loop, print `Count: 1` to `Count: 5`.

Q2. Using `range(5)`, print its values. Then print values of `range(1, 5)` and `range(2, 10, 2)`. Output should match: `0 1 2 3 4`, `1 2 3 4`, `2 4 6 8`.

Q3. Using a `for` loop with `end=' '`, print `1 2 3 4 5` on one line.

Q4. Take a number `n` and print its multiplication table up to 10, e.g. `5 x 1 = 5`.

Q5. Print numbers 1 to 5 and use `break` to stop when the number is 3. Expected output: `1`, `2`.

Q6. Print numbers 1 to 5 and use `continue` to skip 3. Expected output: `1 2 4 5`.

Q7. Write a `for` loop over `range(1, 4)` with an `if i == 2:` block that contains only `pass`. Print `i` after the `if`. Write a comment explaining what `pass` does.

Q8. Using nested loops (outer `i` from 1 to 3, inner `j` from 1 to 3), print `i=1, j=1` and so on, giving 9 lines.

Q9. Print this pattern using nested loops:

```
*
**
***
****
*****
```

Q10. Take a number such as `345`. Using a `while` loop, print the sum of its digits (answer: 12).

Section 6: Functions (Notes Page 6)

Q1. Write `greet(name)` that prints `Hello, <name>!`. Call it with `'QA Insights'`. Add a docstring explaining what it does.

Q2. Write `add(a, b)` that returns `a + b`. Call it with `10, 20` and print the result (30). Write a comment naming the parameters and the arguments.

Q3. Write `multiply(x, y)` that returns the product. Print `multiply(4, 5)`.

Q4. Write a function that prints a message but has no `return`. Store its result in a variable and print the variable. Write a comment on what you see (`None`).

Q5. Write `greet(name, msg='Welcome!')`. Call it as `greet('Alice')` and `greet('Bob', 'Good Morning!')`.

Q6. Write `sum_all(*args)` that returns the sum of all values. Call it with `1, 2, 3, 4, 5` (answer: 15) and with just `10, 20`.

Q7. Write `print_info(**kwargs)` that prints each key and value as `key: value`. Call it with `name='Alice', age=25, city='Hyderabad'`.

Q8. Create a global variable `x = 'Global'`. Inside a function, create a local variable `y = 'Local'` and print both. Outside the function, print `x`. Then try printing `y` outside and write a comment about the error.

Q9. Write `is_even(n)` that returns `True` or `False`. Use it to print all even numbers from 1 to 20.

Q10. Write `factorial(n)` using a loop. Print `factorial(5)` (answer: 120).

Section 7: Strings (Notes Page 7)

Q1. Create `s = 'Python'`. Print `s[0]`, `s[3]`, `s[-1]`, `s[-2]`. Expected: `P`, `h`, `n`, `o`.

Q2. Print the concatenation of `'Hello'` and `'Python'` with a space between. Print `'Hi' * 3`. Print `len('Python')`. Print whether `'Py'` is in `'Python'`.

Q3. Create `s = 'Python Programming'`. Print `s[0:6]`, `s[7:]`, `s[:6]`, `s[::2]`, `s[::-1]`, `s[-11:-1]`. Write a comment beside each describing what it gives.

Q4. For `s = ' hello world '`, print the result of `lower()`, `upper()`, `capitalize()`, `title()`, `strip()`, `lstrip()`, `rstrip()`.

Q5. For `s = 'a_b_c'`, print `s.replace('_', '-')`. For `'hello'`, print `find('e')`. For `'banana'`, print `count('a')`.

Q6. Split `'a,b,c'` using `split(',')` and print the list. Then join `['a', 'b', 'c']` using `'-'.join(...)` and print the result.

Q7. Print `Name: Alice` three times, using `%` formatting, `.format()`, and an f-string.

Q8. Take a name and an age. Print `Hi <name>, you are <age> years old.` using an f-string.

Q9. Take a word and print whether it is a palindrome (reads the same backward), using slicing.

Q10. Take a sentence and print the number of vowels in it.

Section 8: Python Collections (Notes Page 8)

Q1 (List). Create `nums = [10, 20, 30, 40]`. Print the item at index 0, the last item, and `nums[1:3]`.

Q2 (List). Using the same list, do these in order and print the list after each step: `append(50)`, `insert(1, 15)`, `extend([60, 70])`, `remove(20)`, `pop()`.

Q3 (List). Print the list sorted, `len()`, `sum()`, `max()`, and `min()` of `[5, 2, 9, 1, 7]`.

Q4 (Tuple). Create `point = (10, 20, 30)`. Print the first item. Try `point[0] = 5` and write a comment about the error. Print `count(10)` and `index(30)`.

Q5 (Tuple). Unpack `(1, 2, 3)` into variables `a, b, c` and print them.

Q6 (Set). Create a set from `[10, 20, 20, 30, 40, 40]` and print it. Write a comment about what happened to duplicates.

Q7 (Set). For `s = {1, 2, 3}`, use `add(4)`, then `remove(2)`, then `update([7, 8])`. Print the set after each step.

Q8 (Set). For `a = {1, 2, 3, 4}` and `b = {3, 4, 5, 6}`, print the union, intersection, and difference `a - b`.

Q9 (Dict). Create `user = {'name': 'Alice', 'age': 25}`. Print `user['name']` and `user.get('city', 'Not found')`.

Q10 (Dict). Add the key `'city': 'Delhi'`, change `age` to 26, and print the dictionary. Then print `keys()`, `values()`, and `items()`.

Q11 (Dict). Take a word and print how many times each character appears. For `hello`: `{'h': 1, 'e': 1, 'l': 2, 'o': 1}`.

Q12 (Choosing). For each of these, write which collection you would use and why, as a comment: (a) a to-do list, (b) x, y coordinates, (c) removing duplicates from a list, (d) storing a user profile.

Section 9: Comprehensions & Lambda (Notes Page 9)

Q1. Using a list comprehension, create `squares` for numbers 1 to 5. Expected: `[1, 4, 9, 16, 25]`.

Q2. Using a list comprehension with `if`, create a list of even numbers from 1 to 20.

Q3. Using a dictionary comprehension, create `{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}`.

Q4. Using a set comprehension, create the unique even numbers from 1 to 10.

Q5. Write a lambda `add` that adds two numbers. Print `add(5, 3)`. Write a lambda `square` and print `square(4)`.

Q6. For `nums = [1, 2, 3, 4]`, use `map()` with a lambda to double each number. Convert the result to a list and print it. Expected: `[2, 4, 6, 8]`.

Q7. For `nums = [1, 2, 3, 4, 5, 6]`, use `filter()` with a lambda to keep only even numbers. Convert to a list and print it.

Q8. Write the same task (squares of 1 to 5) in three ways: normal `for` loop, list comprehension, and `map()` with lambda. Print all three results.

Section 10: Modules & Packages (Notes Page 10)

Q1. Create `mymodule.py` with `PI = 3.14159` and a function `add(a, b)` that returns `a + b`.

Q2. In a new file in the same folder, use `import mymodule` and print `mymodule.PI` and `mymodule.add(10, 20)`.

Q3. In another file, use `from mymodule import add, PI` and print `PI` and `add(5, 7)`.

Q4. Use `import math` and print `math.sqrt(16)`, `math.pi`, and `math.pow(2, 5)`.

Q5. Use `random.randint(1, 10)` to print a random number 3 times.

Q6. Use `datetime.now()` to print the current date and time.

Q7. Use `os.getcwd()` to print the current folder, and `os.listdir('.')` to print its files. Use `sys.version` to print the Python version.

Q8. Create a folder `mypackage` containing `__init__.py`, `math_ops.py` (with a function `add`), and `string_ops.py` (with a function `shout(text)` that returns `text.upper()`). From a file outside the folder, import and use both functions.

Q9. Write the commands (as comments) to: install `requests`, install `pandas` version `2.1.4`, list installed packages, and uninstall `requests`.

Q10. Write the commands (as comments) to create a virtual environment named `venv`, and to activate it on Windows and on Linux/Mac.

Section 11: File Handling (Notes Page 11)

Q1. Using `with open('notes.txt', 'w')`, write the three lines `Hello QA Insights!`, `Python is awesome!`, `File handling is easy.`, each on its own line.

Q2. Read `notes.txt` completely using `read()` and print it.

Q3. Read `notes.txt` line by line using a `for` loop. Print each line using `strip()`.

Q4. Read the file using `readline()` once and then `readlines()`. Print both results.

Q5. Open `notes.txt` in `'a'` mode and add `New log entry`. Print the file again to confirm the old lines are still there.

Q6. Write a comment explaining the difference between modes `'r'`, `'w'`, `'a'`, and `'x'`.

Q7. Count and print the number of lines and the number of words in `notes.txt`.

Q8. Using the `csv` module, write a file `out.csv` with the header `Name, Age` and 2 rows (`Alice, 25` and `Bob, 30`). Then read it back with `csv.reader` and print each row.

Q9. Using the `json` module, save the dictionary `{'name': 'Alice', 'age': 25}` into `info.json` using `json.dump(..., indent=4)`. Read it back using `json.load()` and print `data['name']`.

Q10. Wrap the code to read a file `missing.txt` in `try-except` so that it prints `File not found` instead of crashing.

Section 12: Exception Handling (Notes Page 12)

Q1. Divide 10 by a number entered by the user. Use `try-except` to catch `ZeroDivisionError` and print `Cannot divide by zero!`.

Q2. Ask the user for an integer. Catch `ValueError` and print `Invalid input! Please enter a number.`

Q3. Combine Q1 and Q2 in one program using two `except` blocks. Add an `else` block that prints `Execution successful.` and a `finally` block that prints `Cleanup: This will run always.`

Q4. Ask for an age. If it is below 18, use `raise ValueError('Age must be 18 or above')`. Otherwise print `Access granted`.

Q5. Create a custom exception class `InvalidAgeError` that inherits from `Exception`. Raise it if the entered age is negative, with the message `Age cannot be negative`.

Q6. Write one small program for each of these errors, catch it, and print a friendly message: `TypeError` (add `5 + 'a'`), `KeyError` (missing dictionary key), `IndexError` (list index too big), `FileNotFoundError`.

Q7. Run `x = 10 / 0` without a `try`. Copy the traceback you get into a comment and underline the line number and the error name.

Q8. Write 3 best practices of exception handling from the notes as comments (for example, avoid bare `except`).

Section 13: Object-Oriented Programming (Notes Page 13)

Q1. Create a class `Car` with a class variable `brand = 'Toyota'` and a method `start()` that prints `Car started`.

Q2. Create two objects `car1` and `car2` of class `Car`. Print `car1.brand` and call `car1.start()`.

Q3. Create a class `Person` with `__init__(self, name, age)` and a method `info()` that prints `Alice is 25 years old`. Create the object `Person('Alice', 25)` and call `info()`.

Q4. Create a class `Student` with `name`, `roll_no`, and `marks`. Add a method `display()` to print all three. Make 3 student objects and call `display()` on each.

Q5 (Inheritance). Create a parent class `Animal` with a method `speak()` printing `Animal speaks`. Create a child class `Dog(Animal)` and call `speak()` on a `Dog` object without writing `speak()` in `Dog`.

Q6 (Polymorphism). Override `speak()` in `Dog` to print `Dog barks`. Write a function `make_sound(obj)` that calls `obj.speak()`. Call it with an `Animal` object and a `Dog` object.

Q7 (Encapsulation). Create a class `Account` with a private attribute `__balance`. Add `deposit(amount)` (only if amount is above 0) and `get_balance()`. Try to print `account.__balance` directly and write a comment on the error.

Q8 (Abstraction). Using `from abc import ABC, abstractmethod`, create an abstract class `Shape` with an abstract method `area()`. Create `Circle(Shape)` that implements `area()` using radius. Try creating a `Shape` object directly and write a comment on the error.

Q9. Write one line each (as comments) defining: Class, Object, Constructor, Inheritance, Polymorphism, Encapsulation, Abstraction.

Section 14: Advanced Python (Notes Page 14)

Q1 (Iterator). Create `nums = [1, 2, 3]`, get `it = iter(nums)`, and print `next(it)` three times. Write a comment on what happens at a fourth `next(it)`.

Q2 (Generator). Write a generator `count(n)` that uses `yield` to give numbers 0 to n-1. Print the values for `count(3)` using a `for` loop.

Q3 (Generator). Write a generator that yields the first 5 even numbers.

Q4 (Decorator). Write a decorator `log_time(func)` that prints `Function started...` before the call and `Function ended...` after it. Apply it using `@log_time` to a function that prints `Hello`.

Q5 (Context manager). Write a program to read a file using a `with` statement and write a comment listing 2 benefits.

Q6 (Built-ins). For `names = ['A', 'B', 'C']` and `scores = [90, 85, 70]`, print the results of `len()`, `sorted(scores)`, `list(enumerate(names))`, `list(zip(names, scores))`, `any([False, True])`, `all([True, False])`, and `isinstance(10, int)`.

Q7. Write 3 performance tips from the notes as comments (for example, use sets for fast membership testing).

Section 15: Python Libraries (Notes Page 15)

For third-party libraries, first install them using `pip install <name>`.

Q1 (os). Print the current folder with `os.getcwd()`. Create a folder `demo` with `os.mkdir('demo')` and print `os.listdir()`.

Q2 (sys). Print `sys.version` and `sys.path`.

Q3 (pathlib). Create `p = Path('demo/file.txt')`. Print `p.name` and `p.exists()`.

Q4 (json). Convert `{'name': 'QA'}` to a JSON string with `json.dumps()`, then back to a dictionary with `json.loads()`. Print both.

Q5 (datetime). Print today's date in the format `2026-10-29` using `strftime('%Y-%m-%d')`.

Q6 (requests). Install `requests`. Call `requests.get('https://api.github.com')`, then print `status_code` and `json()`.

Q7 (NumPy). Install `numpy`. Create `arr = np.array([1, 2, 3])`. Print `arr * 2` and `arr.mean()`.

Q8 (Pandas). Install `pandas`. Create a DataFrame with columns `Name` (`A`, `B`) and `Score` (`90`, `85`). Print the DataFrame and the mean of `Score`.

Q9 (Matplotlib). Install `matplotlib`. Plot `x = [1, 2, 3]` against `y = [10, 20, 15]` with `marker='o'`, add the title `Sample Plot`, and show the graph.

Section 16: Real-World Python Automation (Notes Page 16)

Q1 (File organizer). Create a folder `src` with a few files of different types (`.pdf`, `.jpg`, `.txt`). Write a program that creates a folder for each extension and moves each file into its matching folder. Use `os.listdir`, `os.makedirs(exist_ok=True)`, and `shutil.move`.

Q2 (CSV processor). Create `input.csv` with columns `ID, Name, Score` and 4 rows. Write a program that reads it and writes only the rows with Score above 90 into `output.csv`.

Q3 (Log analyzer). Create `app.log` with 5 lines, some starting with `INFO`, `ERROR`, `WARNING`. Read the file, collect the lines containing `ERROR`, and print `Total Errors: <count>`.

Q4 (REST API). Using `requests`, call `https://jsonplaceholder.typicode.com/users/1` and print the `name` from the JSON response.

Q5 (Backup). Using `shutil.make_archive`, create a zip backup of a folder. Include the date from `datetime.now().strftime('%Y%m%d')` in the backup name.

Q6 (Workflow). Combine Q2 and Q3 steps in a single script: Read a file, process it, and write a report file. Add `try-except` and print a message at each stage.

Section 17: Python Best Practices (Notes Page 17)

Q1. Rewrite this badly named code with proper names and 4-space indentation: `def f(a,b):return a+b` and `X=f(1,2)`.

Q2. Write one example each of snake_case (variable and function), PascalCase (class), and UPPER_SNAKE_CASE (constant).

Q3. Write `add(a, b)` with a proper docstring, then print `add.__doc__`.

Q4. Create a `requirements.txt` with 3 packages and versions. Write the command to install from it as a comment.

Q5. Draw (as a comment) the project structure with `src/`, `tests/`, `requirements.txt`, `README.md`, and `.gitignore`.

Q6. For each bad habit below, write the corrected version: (a) `def fun(lst=[])`, (b) `except: pass`, (c) `API_KEY = '12345'` hardcoded in code, (d) `f = open('file.txt')` without closing.

Final Mini Project: Student Management Menu

Build one program that uses ideas from all sections.

1. Show the menu: `1. Add Student 2. View All 3. Search by Name 4. Save to File 5. Exit`.
2. Store students as a list of dictionaries with `name`, `roll`, and `marks`.
3. Use a separate function for each menu option.
4. Repeat the menu in a `while` loop until the user picks 5.
5. Use `try-except` to handle wrong input (text instead of numbers).
6. For option 4, save the data to `students.json` using the `json` module.
7. Use a list comprehension to show only the students with marks above 75 (add as option 6).
8. Add comments and docstrings, and follow naming conventions from Section 17.

Submission

- Submit 17 section files plus the project file.
- Each file must include sample output at the bottom as comments.
- Programs must run without errors.